

 ASCENT FORMATION

Langage Rust — Jour 4

Et après ? Support de cours

Ascent Formation · Journée complémentaire (7 h) · Marc DESCHAMPS
Version 2026

Sommaire

Avant-propos

Chapitre 1 — Les collections : HashMap et HashSet

- 1.1 HashMap : associer une clé à une valeur
- 1.2 L'API entry
- 1.3 HashSet : un ensemble sans doublon
- 1.4 Choisir sa collection

Chapitre 2 — Traits et génériques

- 2.1 Définir et implémenter un trait
- 2.2 Les traits de la bibliothèque standard
- 2.3 Les génériques
- 2.4 Syntaxes des contraintes
- 2.5 Génériques ou objets de trait

Chapitre 3 — Les durées de vie (lifetimes)

- 3.1 Pourquoi le compilateur demande des annotations
- 3.2 N'annoter que ce qui est lié
- 3.3 Les règles d'élision
- 3.4 Ce qu'aucune annotation ne peut sauver
- 3.5 Structures contenant des références

Chapitre 4 — Itérateurs et fermetures

- 4.1 Les fermetures
- 4.2 Les itérateurs
- 4.3 Écrire son propre itérateur

Chapitre 5 — Concurrency et programmation asynchrone

- 5.1 Les threads
- 5.2 Partager l'état : Arc et Mutex
- 5.3 Communiquer par canaux
- 5.4 async / await et tokio

Chapitre 6 — L'écosystème

- 6.1 crates.io, docs.rs et Cargo
- 6.2 serde : sérialisation
- 6.3 anyhow : des erreurs simples pour les applications
- 6.4 clap : les arguments de la ligne de commande
- 6.5 axum : une API web
- 6.6 D'autres crates à connaître

Annexe — Ressources pour continuer

Avant-propos

Ce support accompagne la quatrième journée, consacrée aux notions qui séparent les bases du Rust « de production » : collections associatives, traits et génériques, durées de vie, itérateurs et fermetures, concurrence, et quelques bibliothèques incontournables de l'écosystème.

L'objectif n'est pas de tout maîtriser en une journée, mais de **savoir lire** du code Rust réel, d'**écrire** de petits programmes qui utilisent ces notions, et de savoir **où chercher** pour aller plus loin.

Les exercices correspondants se trouvent dans le dossier `rust-jour4-exercices/` (ex15 à ex22). Ils fonctionnent comme ceux des jours précédents : les fonctions contiennent `todo!()`, les tests sont fournis, l'objectif est de les faire passer au vert avec `cargo test -p <exercice>`.

Prérequis techniques : les exercices ex20 à ex22 utilisent des crates externes (tokio, serde, clap, anyhow, axum). Il faut un accès à crates.io, ou avoir exécuté une fois `cargo build` dans le dossier pour précharger les dépendances.

Chapitre 1 — Les collections : HashMap et HashSet

1.1 HashMap : associer une clé à une valeur

Un `HashMap<K, V>` associe à chaque clé unique une valeur. Il se trouve dans `std::collections`, et doit donc être importé.

```
use std::collections::HashMap;

fn main() {
    let mut stock: HashMap<String, u32> = HashMap::new();
    stock.insert(String::from("pommes"), 12);
    stock.insert(String::from("poires"), 5);

    match stock.get("pommes") {
        Some(qte) => println!("{qte} pommes"),
        None => println!("pas de pommes"),
    }

    stock.insert(String::from("pommes"), 20); // remplace l'ancienne valeur
    stock.remove("poires");

    for (fruit, qte) in &stock {
        println!("{fruit} : {qte}");
    }
}
```

Points à retenir :

- `get` renvoie une `Option<V>` : la clé peut être absente. L'indexation `stock["pommes"]` existe aussi, mais elle **panique** si la clé est absente.
- `insert` remplace la valeur si la clé existe déjà et renvoie l'ancienne valeur dans une `Option`.
- L'ordre de parcours **n'est pas garanti** et peut changer d'une exécution à l'autre. Pour des clés triées, utiliser `BTreeMap`.
- La table **prend possession** des `String` insérées comme clés ou valeurs.

1.2 L'API entry

Le cas le plus fréquent est « insérer si absent, sinon mettre à jour ». L'API `entry` le fait en un seul accès :

```
use std::collections::HashMap;

fn main() {
    let texte = "le chat voit le chien";
    let mut compteur: HashMap<&str, usize> = HashMap::new();
    for mot in texte.split_whitespace() {
        *compteur.entry(mot).or_insert(0) += 1;
    }
    println!("{:?}", compteur.get("le")); // Some(2)

    let mut groupes: HashMap<char, Vec<&str>> = HashMap::new();
    for mot in ["Rust", "Ruby", "Go"] {
        let initiale = mot.chars().next().unwrap();
        groupes.entry(initiale).or_default().push(mot);
    }
    println!("{:?}", groupes[&'R']); // ["Rust", "Ruby"]
}
```

`or_insert(v)` insère `v` si la clé est absente ; `or_default()` insère la valeur par défaut du type (0, chaîne vide, vecteur vide). Les deux renvoient une référence mutable vers la valeur.

1.3 HashSet : un ensemble sans doublon

Un `HashSet<T>` stocke des valeurs **uniques**, sans ordre. Il répond très vite à la question « cette valeur est-elle présente ? ».

```

use std::collections::HashSet;

fn main() {
    let a: HashSet<i32> = [1, 2, 3].into_iter().collect();
    let b: HashSet<i32> = [2, 3, 4].into_iter().collect();

    println!("{}", a.contains(&2));           // true
    let mut communs: Vec<_> = a.intersection(&b).collect();
    communs.sort();
    println!("{communs:?}");                  // [2, 3]

    let mut vus = HashSet::new();
    println!("{}", vus.insert("x"));           // true : nouveau
    println!("{}", vus.insert("x"));           // false : déjà présent
}

```

Le fait que `insert` renvoie `false` pour un doublon permet de détecter les répétitions en un seul passage.

1.4 Choisir sa collection

Besoin	Collection
Liste ordonnée, accès par indice	<code>Vec<T></code>
Associer une clé à une valeur	<code>HashMap<K, V></code>
Associer, avec des clés triées	<code>BTreeMap<K, V></code>
Unicité, test d'appartenance	<code>HashSet<T></code> (ou <code>BTreeSet<T></code> trié)
File : ajout et retrait aux deux extrémités	<code>VecDeque<T></code>

Pour être utilisé comme clé de `HashMap` ou élément de `HashSet`, un type doit implémenter les traits `Eq` et `Hash`. C'est le cas des entiers, des `char`, des `String` et `&str`, et de vos propres types avec `#[derive(PartialEq, Eq, Hash)]`. Ce n'est **pas** le cas des `f64`.

Chapitre 2 — Traits et génériques

2.1 Définir et implémenter un trait

Un **trait** décrit un comportement que plusieurs types peuvent partager. C'est l'équivalent d'une interface en Java ou en C#, avec en plus la possibilité de fournir des méthodes par défaut.

```
trait Forme {
    fn aire(&self) -> f64;
    fn nom(&self) -> String;

    // Méthode par défaut : elle peut appeler les autres méthodes du trait
    fn description(&self) -> String {
        format!("{ d'aire {:.2}", self.nom(), self.aire())
    }
}

struct Cercle { rayon: f64 }
struct Rectangle { largeur: f64, hauteur: f64 }

impl Forme for Cercle {
    fn aire(&self) -> f64 { std::f64::consts::PI * self.rayon * self.rayon }
    fn nom(&self) -> String { String::from("Cercle") }
}

impl Forme for Rectangle {
    fn aire(&self) -> f64 { self.largeur * self.hauteur }
    fn nom(&self) -> String { String::from("Rectangle") }
    // Redéfinition de la méthode par défaut
    fn description(&self) -> String {
        format!("Rectangle {}x{}", self.largeur, self.hauteur)
    }
}

fn main() {
    println!("{}", Cercle { rayon: 2.0 }.description()); // Cercle d'aire 12.57
    println!("{}", Rectangle { largeur: 3.0, hauteur: 4.0 }.description());
}
```

2.2 Les traits de la bibliothèque standard

Une grande partie de la bibliothèque standard repose sur des traits. Les plus courants s'obtiennent avec `#[derive]` :

Trait	Ce qu'il permet	Obtention
Debug	Affichage <code>{:?}</code>	<code>derive</code>
Clone	Copie explicite avec <code>.clone()</code>	<code>derive</code>
PartialEq	Comparaison <code>==</code>	<code>derive</code>
PartialOrd	Comparaison <code><</code> , <code>></code>	<code>derive</code>
Default	Valeur par défaut <code>T::default()</code>	<code>derive</code>
Display	Affichage <code>{}</code> et <code>.to_string()</code>	à écrire

```
use std::fmt;

#[derive(Debug, Clone, PartialEq)]
struct Point { x: f64, y: f64 }

impl fmt::Display for Point {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}

fn main() {
    let p = Point { x: 1.0, y: 2.5 };
    println!("{}", p); // (1, 2.5)
    println!("{}", p); // Point { x: 1.0, y: 2.5 }
}
```

2.3 Les génériques

Une fonction générique fonctionne pour plusieurs types. Le paramètre de type `T` doit être **contraint** par les traits dont la fonction a besoin :

```
fn plus_grand<T: PartialOrd + Copy>(liste: &[T]) -> Option<T> {
    let mut max = *liste.first()?;
    for &x in liste {
        if x > max {
            max = x;
        }
    }
    Some(max)
}

fn main() {
    println!("{:?}", plus_grand(&[3, 9, 2])); // Some(9)
    println!("{:?}", plus_grand(&[1.5, -2.0])); // Some(1.5)
    println!("{:?}", plus_grand(&['a', 'z'])); // Some('z')
}
```

Sans `PartialOrd`, la comparaison `x > max` est refusée : le compilateur ne sait pas si `T` est comparable. Sans `Copy`, on ne peut pas sortir `x` de la tranche par copie.

Les structures et leurs `impl` peuvent aussi être génériques :

```
struct Paire<T> { a: T, b: T }

impl<T: PartialOrd> Paire<T> {
    fn max(&self) -> &T {
        if self.a >= self.b { &self.a } else { &self.b }
    }
}
```

`Option<T>`, `Result<T, E>`, `Vec<T>` et `HashMap<K, V>` sont eux-mêmes des types génériques.

2.4 Syntaxes des contraintes

Trois écritures équivalentes, à choisir selon la lisibilité :

```
fn fiche<T: Forme>(f: &T) -> String { f.description() }

fn fiche2(f: &impl Forme) -> String { f.description() }

fn fiche3<T>(f: &T) -> String
where
    T: Forme + std::fmt::Debug,
{
    format!("{:?}", f)
}
```

`impl Trait` peut aussi apparaître en **type de retour** : la fonction renvoie « un certain type qui implémente ce trait », sans le nommer. C'est indispensable pour renvoyer une fermeture (chapitre 4).

2.5 Génériques ou objets de trait

Une fonction générique est **compilée une fois par type utilisé** (monomorphisation) : c'est très rapide, mais une même liste ne peut contenir qu'un seul type.

Pour stocker des types **différents** dans une même collection, on utilise un **objet de trait** `dyn Trait`, placé derrière un pointeur (`Box`, `&`) car sa taille n'est pas connue à la compilation :

```
fn aire_totale(formes: &[Box<dyn Forme>]) -> f64 {
    formes.iter().map(|f| f.aire()).sum()
}

let formes: Vec<Box<dyn Forme>> = vec![
    Box::new(Cercle { rayon: 1.0 }),
    Box::new(Rectangle { largeur: 2.0, hauteur: 3.0 }),
];
```

L'appel de méthode passe alors par une table de fonctions (répartition dynamique), comme les méthodes virtuelles en C++ ou en Java.

Chapitre 3 — Les durées de vie (lifetimes)

3.1 Pourquoi le compilateur demande des annotations

Le vérificateur d'emprunts garantit qu'aucune référence ne survit à la donnée qu'elle désigne. Pour une fonction qui reçoit **plusieurs** références et en renvoie une, il ne peut pas deviner de laquelle provient le résultat :

```
fn plus_longue(a: &str, b: &str) -> &str {    // error[E0106]: missing lifetime specifier
    if a.len() > b.len() { a } else { b }
}
```

On nomme alors la relation avec un paramètre de durée de vie, noté `'a` :

```
fn plus_longue<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() > b.len() { a } else { b }
}
```

Lecture : « le résultat est valide **au plus aussi longtemps** que la plus courte des durées de vie de `a` et `b` ». Une annotation **ne modifie pas** la durée de vie des données ; elle **décrit** une relation que le compilateur vérifie ensuite à chaque appel.

3.2 N'annoter que ce qui est lié

Si le résultat ne dépend que d'un paramètre, seul celui-ci est annoté :

```
fn apres_prefixe<'a>(texte: &'a str, prefixe: &str) -> &'a str {
    texte.strip_prefix(prefixe).unwrap_or(texte)
}
```

L'appelant peut alors libérer `prefixe` tout en conservant le résultat. Annoter les deux paramètres avec `'a` compilerait aussi, mais serait inutilement restrictif.

3.3 Les règles d'élision

Dans la plupart des cas, aucune annotation n'est nécessaire, grâce aux **règles d'élision** :

1. Chaque référence en paramètre reçoit sa propre durée de vie.
2. S'il n'y a **qu'une seule** référence en entrée, le résultat reçoit sa durée de vie.
3. Dans une méthode avec `&self` ou `&mut self`, le résultat reçoit la durée de vie de `self`.

```
fn premier_mot(texte: &str) -> &str {    // règle 2 : aucune annotation
    texte.split_whitespace().next().unwrap_or("")
}
```

3.4 Ce qu'aucune annotation ne peut sauver

Renvoyer une référence vers une valeur **créée dans la fonction** est impossible : cette valeur est détruite à la sortie de la fonction.

```
fn salutation(nom: &str) -> &str {
    let message = format!("Bonjour {nom} !");
    &message    // error[E0515]: cannot return reference to local variable
}
```

La solution est de renvoyer la valeur **possédée** : `fn salutation(nom: &str) -> String`.

3.5 Structures contenant des références

Une structure qui contient une référence doit déclarer une durée de vie, qui garantit qu'elle ne survivra pas à la donnée empruntée :

```

struct Extrait<'a> {
    texte: &'a str,
}

impl<'a> Extrait<'a> {
    fn premiere_phrase(&self) -> &'a str {
        match self.texte.find('.') {
            Some(i) => &self.texte[..i],
            None => self.texte,
        }
    }
}

```

Le type de retour `&'a str` (et non `&str`, qui serait lié à `self` par la règle 3) indique que la phrase emprunte le **texte d'origine** : elle reste utilisable après la destruction de l'`Extrait`.

Conseil pour débiter : dans vos structures, préférez des champs possédés (`String`, `Vec<T>`). Réservez les références aux paramètres de fonctions. Vous rencontrerez surtout les durées de vie en **lisant** du code et des signatures de bibliothèques. La durée de vie spéciale `'static` désigne une donnée valide pendant tout le programme, comme les littéraux de chaîne.

Chapitre 4 — Itérateurs et fermetures

4.1 Les fermetures

Une **fermeture** (*closure*) est une fonction anonyme qui peut **capturer** les variables de son environnement :

```
fn main() {
    let double = |x: i32| x * 2;
    let tva = 1.2;
    let ttc = |ht: f64| ht * tva;           // capture tva
    println!("{}", double(4), ttc(100.0));
}
```

Les types des paramètres sont généralement déduits. Selon ce qu'elle fait de ses captures, une fermeture implémente un ou plusieurs traits :

Trait	La fermeture...	Appelable
<code>Fn</code>	lit ses captures	autant de fois que voulu
<code>FnMut</code>	modifie ses captures	autant de fois que voulu, si <code>mut</code>
<code>FnOnce</code>	consomme ses captures	une seule fois

Le mot-clé `move` force la fermeture à **prendre possession** de ses captures : indispensable pour la renvoyer depuis une fonction ou l'envoyer dans un thread.

```
fn appliquer<F: Fn(f64) -> f64>(valeurs: &[f64], f: F) -> Vec<f64> {
    valeurs.iter().map(|&x| f(x)).collect()
}

fn creer_remise(taux: f64) -> impl Fn(f64) -> f64 {
    move |prix| prix * (1.0 - taux / 100.0)
}

fn creer_compteur() -> impl FnMut() -> u32 {
    let mut n = 0;
    move || {
        n += 1;
        n
    }
}

fn main() {
    let soldes = creer_remise(20.0);
    println!("{}", appliquer(&[50.0, 10.0], soldes)); // [40.0, 8.0]
    let mut c = creer_compteur();
    println!("{}", c(), c());                        // 1 2
}
```

4.2 Les itérateurs

Un itérateur produit une séquence de valeurs. Trois façons de l'obtenir depuis une collection :

Méthode	Éléments produits	La collection...
<code>iter()</code>	<code>&T</code>	est empruntée
<code>iter_mut()</code>	<code>&mut T</code>	est empruntée en écriture
<code>into_iter()</code>	<code>T</code>	est consommée

Les **adaptateurs** transforment un itérateur en un autre ; les **consommateurs** produisent un résultat final.

```
fn main() {
    let notes = vec![12, 8, 15, 17, 9];

    let reussites: Vec<i32> = notes.iter().filter(|&n| n >= 10).copied().collect();
    let somme: i32 = notes.iter().sum();
    let meilleure = notes.iter().max();
    let rang = notes.iter().position(|&n| n == 15);
    let toutes_valides = notes.iter().all(|&n| n <= 20);

    println!("{reussites:?} {somme} {meilleure:?} {rang:?} {toutes_valides}");
    // [12, 15, 17] 61 Some(17) Some(2) true

    for (i, n) in notes.iter().enumerate().skip(1).take(2) {
        println!("note {i} : {n}");
    }
}
```

Les itérateurs sont **paresseux** : une chaîne d’adaptateurs ne fait rien tant qu’un consommateur (`collect` , `sum` , `count` , une boucle `for...`) ne la parcourt pas. Ils sont compilés en code aussi efficace qu’une boucle écrite à la main : c’est une **abstraction à coût nul** .

Adaptateurs	Consommateurs
<code>map</code> , <code>filter</code> , <code>filter_map</code>	<code>collect</code> , <code>sum</code> , <code>count</code>
<code>enumerate</code> , <code>zip</code> , <code>rev</code> , <code>chain</code>	<code>min</code> , <code>max</code> , <code>max_by</code> , <code>min_by_key</code>
<code>take</code> , <code>skip</code> , <code>take_while</code>	<code>find</code> , <code>position</code>
<code>flat_map</code> , <code>copied</code> , <code>cloned</code>	<code>any</code> , <code>all</code> , <code>fold</code>

Les flottants ne sont pas Ord (à cause de NaN) : `max()` ne fonctionne pas sur des `f64`. On utilise `max_by(|a, b| a.total_cmp(b))`.

4.3 Écrire son propre itérateur

Il suffit d’implémenter le trait `Iterator`, c’est-à-dire une seule méthode, `next` :

```
struct Fibonacci { courant: u64, suivant: u64 }

impl Iterator for Fibonacci {
    type Item = u64;
    fn next(&mut self) -> Option<u64> {
        let valeur = self.courant;
        self.courant = self.suivant;
        self.suivant += valeur;
        Some(valeur)
    }
}

fn main() {
    let fib = Fibonacci { courant: 0, suivant: 1 };
    let v: Vec<u64> = fib.take(10).collect();
    println!("{v:?}"); // [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
}
```

Tous les adaptateurs et consommateurs sont alors disponibles gratuitement : ce sont des méthodes par défaut du trait `Iterator`.

Chapitre 5 — Concurrency et programmation asynchrone

5.1 Les threads

```
use std::thread;

fn main() {
    let donnees = vec![1, 2, 3, 4];
    let h = thread::spawn(move || donnees.iter().sum::<i32>());
    println!("{}", h.join().unwrap()); // 10
}
```

`thread::spawn` lance un thread système ; `join` attend sa fin et récupère son résultat. La fermeture doit être `move` : le thread peut survivre à la fonction qui l'a créé, il doit donc posséder ses données.

C'est ici que le système de possession montre toute sa valeur : **les accès concurrents non synchronisés (data races) sont des erreurs de compilation**. On parle de *fearless concurrency*.

5.2 Partager l'état : Arc et Mutex

Pour que plusieurs threads modifient une même donnée, on combine :

- `Arc<T>` (*Atomically Reference Counted*) : plusieurs propriétaires d'une même donnée, utilisable entre threads ;
- `Mutex<T>` : un verrou qui garantit un seul accès à la fois ; `lock()` renvoie un garde, libéré automatiquement à la fin de sa portée.

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let compteur = Arc::new(Mutex::new(0));
    let mut handles = Vec::new();
    for _ in 0..8 {
        let c = Arc::clone(&compteur);
        handles.push(thread::spawn(move || {
            *c.lock().unwrap() += 1;
        }));
    }
    for h in handles {
        h.join().unwrap();
    }
    println!("{}", *compteur.lock().unwrap()); // 8
}
```

5.3 Communiquer par canaux

Plutôt que de partager de la mémoire, les threads peuvent **s'envoyer des messages** :

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();
    thread::spawn(move || {
        for i in 1..=3 {
            tx.send(i * i).unwrap();
        }
    }); // tx est détruit ici : le canal se ferme
    let recus: Vec<i32> = rx.iter().collect();
    println!("{recus:?}"); // [1, 4, 9]
}
```

`thread::scope` (depuis Rust 1.63) permet aussi de lancer des threads qui **empruntent** des données locales, sans `move` : la portée garantit qu'ils sont tous terminés avant la fin du bloc.

5.4 async / await et tokio

Les threads conviennent au **calcul intensif**. Pour des programmes qui passent leur temps à **attendre** (requêtes réseau, base de données, fichiers), on utilise la programmation asynchrone : des milliers de tâches légères partagent quelques threads.

```

use std::time::Duration;
use tokio::time::sleep;

async fn traitement(x: u64) -> u64 {
    sleep(Duration::from_millis(100)).await; // rend la main pendant l'attente
    x * 2
}

#[tokio::main]
async fn main() {
    let a = tokio::spawn(traitement(1));
    let b = tokio::spawn(traitement(2));
    println!("{}", a.await.unwrap(), b.await.unwrap()); // 2 4, en 100 ms au total
}

```

Points clés :

- une fonction `async` renvoie un *future* : rien ne s'exécute tant qu'on ne fait pas `.await` (ou `tokio::spawn`) ;
- Rust ne fournit pas d'exécuteur dans la bibliothèque standard : **tokio** est le runtime de référence ;
- `tokio::select!` attend le premier de plusieurs futures, `tokio::time::timeout` limite une durée ;
- ne jamais appeler d'opération bloquante (`std::thread::sleep`, gros calcul) dans une fonction `async` : cela bloque tout le runtime.

Threads	async
Calcul intensif (CPU)	Attente d'entrées/sorties
Quelques dizaines	Des milliers de tâches
Bibliothèque standard	Un runtime : tokio

Chapitre 6 — L'écosystème

6.1 crates.io, docs.rs et Cargo

Les bibliothèques Rust (*crates*) sont publiées sur **crates.io** et documentées automatiquement sur **docs.rs**. On les ajoute avec `cargo add` :

```
cargo add serde --features derive
cargo add serde_json anyhow
cargo add clap --features derive
cargo add tokio --features full
```

Les *features* activent des parties optionnelles d'une crate. `Cargo.lock` fige les versions exactes : on le versionne pour une application, pas pour une bibliothèque.

6.2 serde : sérialisation

serde convertit vos types depuis et vers de nombreux formats (JSON avec `serde_json`, mais aussi TOML, YAML, CSV...), grâce à deux `derive` :

```
use serde::{Deserialize, Serialize};

#[derive(Debug, Serialize, Deserialize)]
struct Tache {
    id: u32,
    titre: String,
    faite: bool,
}

fn main() -> Result<(), serde_json::Error> {
    let t = Tache { id: 1, titre: "Lire le Book".into(), faite: false };
    let json = serde_json::to_string(&t)?;
    println!("{}", json); // {"id":1,"titre":"Lire le Book","faite":false}
    let relue: Tache = serde_json::from_str(&json)?;
    println!("{}", relue);
    Ok(())
}
```

6.3 anyhow : des erreurs simples pour les applications

Dans une application, on veut surtout propager les erreurs avec un message clair. `anyhow::Result<T>` accepte n'importe quelle erreur avec `?`, et `with_context` ajoute une explication :

```
use anyhow::{bail, Context, Result};
use std::fs;

fn lire_config(chemin: &str) -> Result<String> {
    let contenu = fs::read_to_string(chemin)
        .with_context(|| format!("lecture de {chemin} impossible"))?;
    if contenu.trim().is_empty() {
        bail!("{chemin} est vide");
    }
    Ok(contenu)
}
```

Avec `fn main() -> anyhow::Result<()>`, une erreur affiche son message et ses causes, puis le programme se termine avec un code d'erreur. Pour une **bibliothèque**, on préfère définir ses propres types d'erreur, éventuellement avec la crate `thiserror`.

6.4 clap : les arguments de la ligne de commande

clap génère l'analyse des arguments, les messages d'erreur et l'aide `--help` à partir d'une structure :

```

use clap::{Parser, Subcommand};

/// Gestionnaire de tâches minimaliste
#[derive(Parser)]
#[command(version, about)]
struct Cli {
    /// Fichier de stockage
    #[arg(short, long, default_value = "taches.json")]
    fichier: String,
    #[command(subcommand)]
    commande: Commande,
}

#[derive(Subcommand)]
enum Commande {
    /// Ajoute une tâche
    Ajouter { titre: String },
    /// Marque une tâche comme faite
    Terminer { id: u32 },
}

fn main() {
    let cli = Cli::parse();
    match cli.commande {
        Commande::Ajouter { titre } => println!("ajout de {titre} dans {}", cli.fichier),
        Commande::Terminer { id } => println!("tâche {id} terminée"),
    }
}

```

Les commentaires `///` deviennent les textes d'aide.

6.5 axum : une API web

axum, construit sur tokio, est l'un des frameworks web les plus utilisés. Un gestionnaire de route est une fonction `async` dont les paramètres sont des **extracteurs** : axum décode la requête d'après leurs types.

```

use axum::{extract::{Path, State}, http::StatusCode, routing::get, Json, Router};
use std::sync::{Arc, Mutex};

type Etat = Arc<Mutex<Vec<String>>>;

async fn detail(State(etat): State<Etat>, Path(id): Path<usize>)
-> Result<Json<String>, StatusCode> {
    etat.lock().unwrap().get(id).cloned().map(Json).ok_or(StatusCode::NOT_FOUND)
}

#[tokio::main]
async fn main() {
    let etat: Etat = Arc::new(Mutex::new(vec!["Rust".into()]));
    let app = Router::new()
        .route("/sante", get(|| async { "ok" })))
        .route("/elements/{id}", get(detail))
        .with_state(etat);
    let ecouteur = tokio::net::TcpListener::bind("127.0.0.1:3000").await.unwrap();
    axum::serve(ecouteur, app).await.unwrap();
}

```

6.6 D'autres crates à connaître

Domaine	Crates
Client HTTP	request
Bases de données	sqlx, diesel
Journalisation	tracing, log
Erreurs de bibliothèque	thiserror
Parallélisme de données	rayon
Tests de propriétés	proptest
Interfaces graphiques	egui, tauri

Annexe — Ressources pour continuer

Ressource	Description
The Rust Programming Language (« le Book »)	La référence officielle, gratuite, en ligne : doc.rust-lang.org/book
Rust by Example	Les notions du langage par des exemples exécutables : doc.rust-lang.org/rust-by-example
Rustlings	Petits exercices à corriger, guidés par le compilateur : github.com/rust-lang/rustlings
Exercism (piste Rust)	Exercices progressifs, avec mentorat bénévole : exercism.org/tracks/rust
Comprehensive Rust	Le cours de Google, du débutant à la concurrence : google.github.io/comprehensive-rust
docs.rs	La documentation de toutes les crates publiées
This Week in Rust	Lettre d'information hebdomadaire de la communauté

Parcours conseillé après la formation : terminer les exercices bonus du kit, faire Rustlings en entier, relire les chapitres 8 à 17 du Book, puis écrire un petit outil personnel (script de traitement de fichiers, petite API) en s'appuyant sur les crates du chapitre 6.